

Smart Electronics Lab v1.2

Guided breadboard learning system, communications console, and SDR
workstation roadmap

Prepared for Bob Litt
Revised June 2026 — Consensus Review Pass

v1.2 · supersedes v1.1

0. What changed in v1.2

v1.2 integrates a second-reviewer (ChatGPT) consensus pass on v1.1. All 16 items were accepted — none were rejected. A few formalize things v1.1 already implied rather than introducing new scope; those are noted as such. Each row below maps the reviewer's item number (R#) to where it landed.

#	Item	Status	Where
R1	3.3V logic warning for Elecrow/ESP32-S3 pins	Accepted — new	Sec. 5, 11
R2	3.3V as the default teaching rail	Accepted — new (added LED-color rationale)	Sec. 5, 12
R3	INA219 measurement topology diagram	Accepted — new	Sec. 5
R4	USB-C meter measures the power path, not branch current	Accepted — new	Sec. 9 (V0)
R5	L8.5 → controlled fault-screen drill, not a real fault	Accepted — fixes a v1.1 flaw	Sec. 12 (L10)
R6	Current-limit table for early lessons	Accepted — new	Sec. 11
R7	Safety/measurement fields in the lesson schema	Accepted — new	Sec. 8
R8	Power-off-before-rewiring as a schema behavior, not prose	Accepted — new	Sec. 8
R9	Avoid high-frequency flash writes	Accepted — new	Sec. 8
R10	Concrete v1 storage plan (LittleFS)	Accepted — names a mechanism v1.1 left generic	Sec. 8
R11	Don't assume Elecrow's connector is Qwiic/STEMMA-compatible	Accepted — new risk	Sec. 4.4, 15
R12	Minimum first-boot test before any breadboard work	Accepted — merged into L0	Sec. 12 (L0)
R13	Concrete Stage 3 control-message examples	Accepted — new	Sec. 4.2
R14	Explicit decision branch for Stage 2b audio risk	Accepted — new	Sec. 4.1
R15	"Product realism" note	Accepted — restates v1.1 Sec. 14 as an explicit callout, not new content	Sec. 14

R16	Decision log (accepted / open)	Accepted — new, moved to front	Sec. 1
-----	--------------------------------	--------------------------------	--------

Net effect: the lesson sequence is renumbered L0–L11 (cleaner than v1.1's L–1...L9+L8.5) because R12 and R5 both touched lesson boundaries enough that patching the old numbering in place would have been more confusing than renumbering once.

1. Decision log NEW

Accepted

- Standard breadboard, not a proprietary breadboard.
- Smartness begins in the power path (V0 → V3 staging, Sec. 9).
- Elecrow is tutor and control head across all three stages.
- Stage 2 split into guided build (2a) and appliance console (2b).
- Stage 3 compute moves to a Raspberry Pi or mini PC; Elecrow stays the control head.
- USB-serial JSON-per-line is the default Stage 3 control interface (Sec. 4.2).
- 3.3V is the default teaching rail; 5V is an explicit later variation, never the default near GPIO/I2C pins.
- Lesson safety and measurement assumptions are schema fields, not prose (Sec. 8).
- Fault-response practice (L10) uses a software-lowered threshold on an already-safe circuit, never a real over-current condition.

Open

- Elecrow expansion connector type and exact pinout (Sec. 4.4, 15).
- Elecrow I2S/audio feasibility (Sec. 4.1).
- Whether lesson content ships compiled into firmware or via LittleFS for v1 (Sec. 8).
- Exact first firmware environment, board definition, and library versions (recorded at L0, Sec. 12).
- INA219 I2C bus logic-side voltage — assumed 3.3V until verified against hardware (Sec. 5).

2. Executive decision

- The project is a guided electronics learning product, not a generic electronics kit.
- The Elecrow rotary display is the front-panel tutor. It displays steps, accepts knob/touch input, and becomes the control head for the communications console and later the SDR workstation.

- The breadboard stays standard. Smart behavior comes from power measurement and software, not from a proprietary smart breadboard.
- The first smart version uses off-the-shelf parts: a USB-C breadboard power supply, an inline USB-C power meter for visible measurement, and an I2C current/voltage sensor the Elecrow can read.
- **3.3V is the default supply rail for lessons that touch Elecrow pins (Sec. 5); 5V is reserved for isolated experiments or an explicit later variation.**

Short version: Buy the BusBoard BB830, USB-C power module, USB-C power meter, Adafruit INA219 current sensor, Qwiic/STEMMA cable kit, jumper wires, and a small set of components. Do not buy a giant Arduino kit.

3. Three-stage architecture

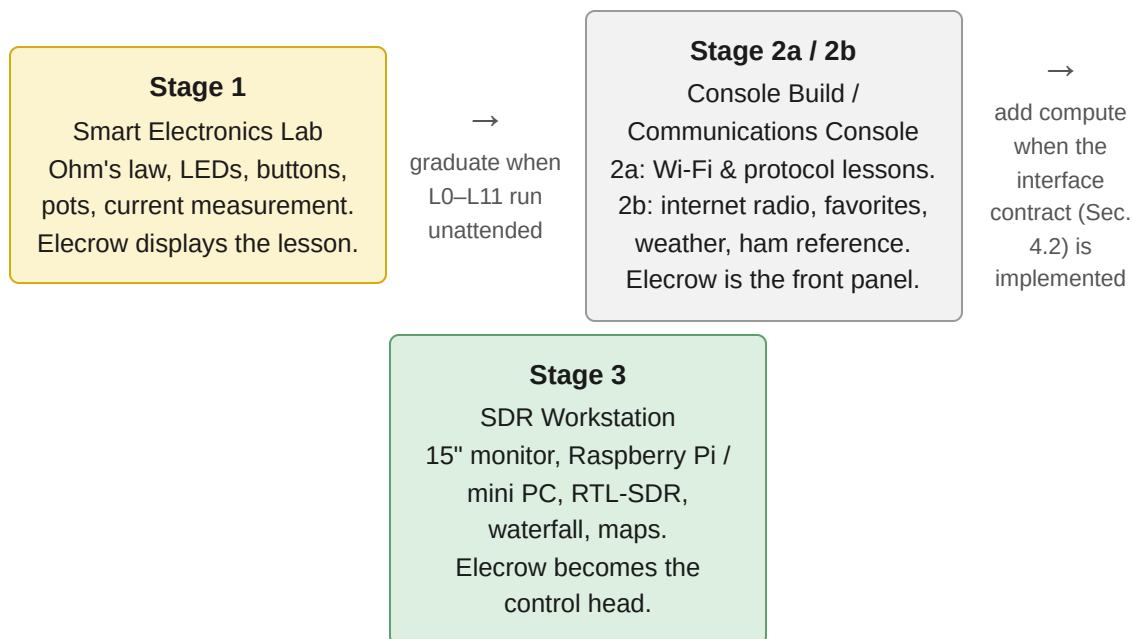


Figure 1. Three-stage roadmap with graduation criteria (stages unchanged from v1.0/v1.1).

Design rule (unchanged): every early purchase remains useful later. The breadboard teaches and prototypes. It does not become the final enclosure.

Stage 1: Smart Electronics Lab

Learn Ohm's law, LEDs, resistors, buttons, potentiometers, basic transistor switching, and simple audio. The display presents instructions and the smart power path measures whether the circuit behaves as expected.

Exit criteria: L0–L11 (Section 12) run end-to-end with the INA219 in the loop, and at least one beginner other than the builder completes the sequence without external help beyond the on-screen tutor.

Stage 2a: Guided Console Build

Same front panel and breadboard. New lessons teach Wi-Fi association, fetching a JSON API (e.g., weather), and basic protocol concepts — still using the instruction-plus-measurement loop, just measuring network/software state instead of only voltage and current.

Stage 2b: Communications Console

Internet radio, favorites, weather screens, local reference screens, basic ham information, and eventually an audio path. This is an appliance feature set built on the Stage 1/2a hardware and skills — not itself a measurement-feedback lesson.

Exit criteria: a short demo of the console playing an internet radio station from a saved favorite, on USB-C power, with the same Elecrow unit used in Stage 1.

Stage 3: SDR Workstation

Add a Raspberry Pi or mini PC, the existing 15" monitor, and an RTL-SDR receiver. The heavy compute handles FFT, waterfall display, maps, decoding, and larger UIs. The Elecrow remains the control head.

Exit criteria: the Elecrow's knob and touch input visibly drive a parameter (e.g., tuned frequency or zoom) in an SDR application running on the separate compute, over the interface defined in Section 4.2.

4. Open risks to resolve before Stage 2

None of these block ordering the Stage 1 parts list. They block committing Stage 2 scope.

4.1 Audio path feasibility DECISION BRANCH ADDED

Stage 2b assumes the Elecrow can stream and play internet radio. The confirmed spec (Section 15) lists UART, I2C, FPC expansion, 16M flash, and 8M PSRAM — it does not confirm an I2S output or amplifier.

Decision branch:

- **If the Elecrow exposes I2S cleanly:** try ESP32 internet radio playback with an add-on I2S DAC/amp module (e.g., MAX98357A class).

- **If it does not, or decoding is unstable** under the touch UI + Wi-Fi load: move audio playback to the Raspberry Pi / mini-PC layer earlier than Stage 3, and keep the Elecrow as front-panel control, station selector, and status display only.

This keeps the internet-radio feature from depending on unverified ESP32 audio capability.

4.2 Stage 1 → Stage 3 control interface contract EXAMPLES ADDED

"The Elecrow remains the control head" is a conclusion, not yet a design. The V1 contract is USB-serial, JSON-per-line, over the existing USB-C data link — no new hardware required. Example messages:

```
{"event": "knob", "delta": 1}
{"event": "button", "name": "select", "state": "pressed"}
{"event": "touch", "x": 214, "y": 93}
{"command": "set_screen", "mode": "sdr", "frequency_hz": 145500000}
```

Stage 2 firmware's internal event handling should already emit knob/touch/button events in this shape, even though nothing consumes them until Stage 3. That avoids a rewrite when the Pi / SDR workstation appears.

4.3 Warn-vs-shutoff fault behavior at V1

Section 9 (V1, software-observed power) can detect an over-current fault via the INA219 but cannot cut power. The exact V1 fault behavior is defined in Section 9, not left implicit.

4.4 Elecrow expansion connector compatibility NEW

The BOM includes a Qwiic/STEMMA cable kit for the INA219 side. Do not assume the Elecrow's own expansion connector is directly Qwiic/STEMMA-compatible — that has not been confirmed against the actual board. See Section 15 for the concrete confirmation steps before wiring the INA219 to the Elecrow.

5. Stage 1 hardware design REVISED

3.3V logic warning: the Elecrow / ESP32-S3 GPIO and I2C pins are 3.3V logic. Do not connect 5V directly to any GPIO, I2C, UART, touch, display, or expansion pin unless the Elecrow documentation explicitly states that pin is 5V tolerant. The breadboard may use 5V for isolated LED/resistor experiments that never touch an Elecrow pin — but any circuit connected to Elecrow data pins defaults to 3.3V logic.

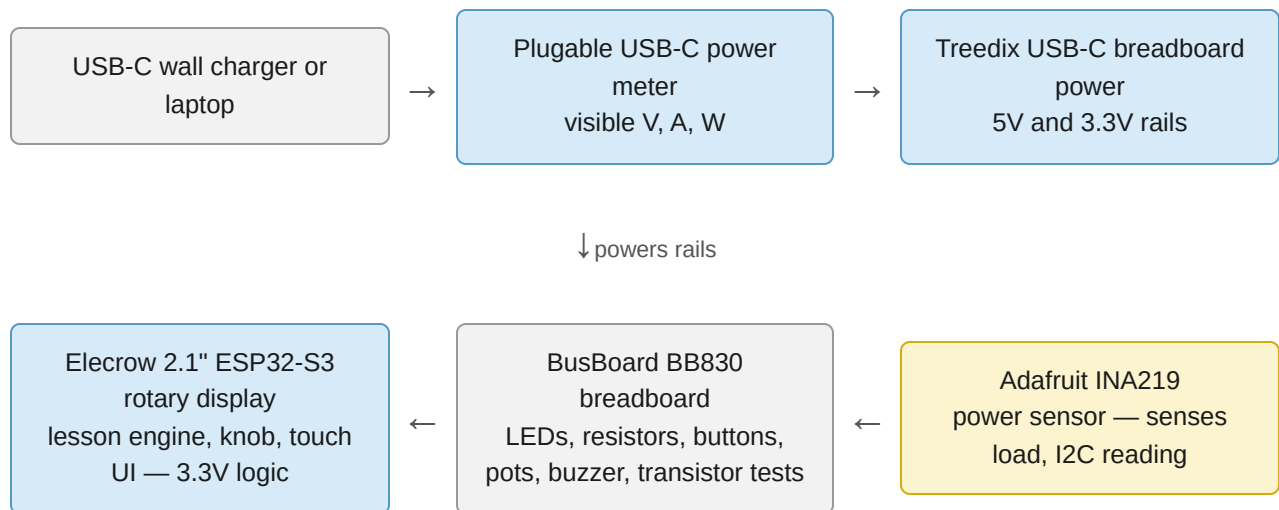


Figure 2. Stage 1 hardware architecture (unchanged from v1.0/v1.1; logic-level note added).

Version 0 is human-observed with the USB-C meter. Version 1 is software-observed with the INA219. Version 2 becomes a custom Smart Power Module.

Mounting answer

The Elecrow display does not need to sit in the breadboard. It sits next to the BB830 on the desk or mat, then connects to the breadboard through its GPIO/I2C connector or pigtail. The breadboard is for temporary circuits. The display is the tutor and controller.

Smart board answer

Do not start by designing a proprietary smart breadboard. Start by making the power path smart. If the Elecrow can read voltage and current from an INA219 sensor over I2C, the lesson engine can compare expected and measured behavior.

INA219 measurement topology NEW

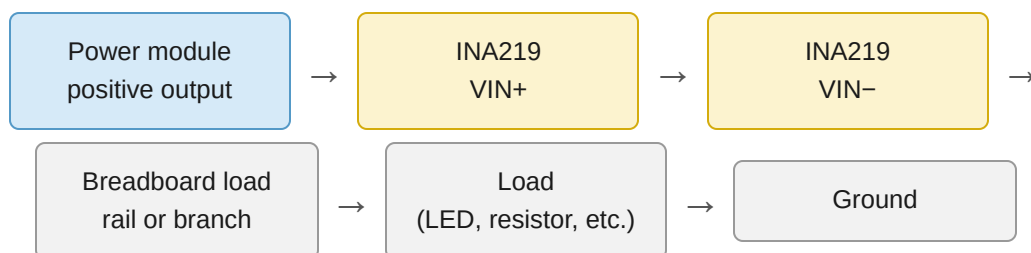


Figure 3. Correct INA219 placement — in series with the measured load, not in parallel with it.

- The INA219 must be in the measured load's current path. If the load can draw current through any path that bypasses the INA219, the software cannot detect that current.
- The Elecrow, INA219, and breadboard circuit must share a common ground.
- The INA219's I2C logic side should be powered at the Elecrow-safe logic voltage — normally 3.3V — unless the Elecrow's I2C pins are confirmed 5V tolerant.

6. Bill of materials

Durable specs live in the Interface/Spec column; ASINs and links are sourcing references only (Section 15), since listings rot.

#	Category	Item	Interface / Spec	Qty / Timing	Rationale
1	Already bought	ELECROW ESP32 Rotary Display, 2.1" 480x480 IPS round, knob + touch	ESP32-S3, Wi-Fi/BLE, UART, I2C, FPC expansion, 16M flash, 8M PSRAM, 5V/1A input, 3.3V logic on GPIO/I2C	1 / done	Front-panel tutor and future control head. Keep as main UI.
2	Core lab	BusBoard BB830 solderless breadboard	830 tie points, 4 power rails, 36V/2A rating, 21–26 AWG	1 / Buy now	Premium standard breadboard, correct size for lessons.
3	Power	Treedix USB-C breadboard power module	USB-C in, 3.3V/5V regulated rail out	1 pack / Buy now	First version of the lab power source. 3.3V rail is the default for Elecrow-adjacent lessons.
4	Visible measurement	Plugable USB-C Power Meter Tester	Inline pass-through, OLED V/A/W readout	1 / Buy now	Makes Ohm's law visible immediately (V0). Measures the USB-C input path, not a specific breadboard branch — see Section 9.
5	Smart measurement	Adafruit INA219 current sensor, STEMMA QT	High-side DC, 26V target, $\pm 3.2A$, I2C, address-selectable	1 / Buy now	First real smart feedback module (V1). Must sit in series with the load — see Figure 3.
6	Smart wiring	Qwiic/STEMMA QT cable kit with	JST-SH 4-pin to breadboard pins	1 / Buy now	Connects STEMMA/Qwiic sensors to the

		breadboard jumpers			breadboard. Confirm the Elecrow's own connector before assuming it's Qwiic-compatible (Section 4.4, 15).
7	Wiring	Silicone flexible Dupont jumper wires	M-M, M-F, F-F	1 kit / Buy now	Far less stiff/annoying than standard kit wires.
8	Resistors	Aniann 1% metal film resistor assortment	1280 pcs, broad E-series coverage	1 / Buy now	Needed for every lesson.
9	LEDs	BOJACK 5mm LED assortment	5 colors	1 / Buy now	Makes current, polarity, and resistance visible. Default lessons use red/yellow at 3.3V — see Section 12.
10	Buttons	BOJACK tactile pushbutton assortment	10 values	1 / Buy now	Input lessons, later physical controls.
11	Potentiometers	MCIGICM 10K breadboard trim pots with knobs	10 pcs, 10K Ω	1 / Buy now	Voltage dividers, analog input, dimming.
12	Capacitors	EMGTMS ceramic capacitor kit	24 values, 480 pcs, 10pF–10 μ F	1 / Buy soon	RC timing, signal smoothing. Not needed for L2.
13	Electrolytic capacitors	Hilitchi electrolytic capacitor assortment	300 pcs	1 / Buy soon	Power smoothing, later audio work.
14	Transistors	PLUSIVO BJT assortment	NPN and PNP	1 / Buy soon	Switching larger loads from a GPIO pin.
15	Sound	Passive/active buzzer module pack	Arduino/ESP32 compatible	1 / Buy soon	Digital output, PWM/tone, audio cues. Drive via transistor, not directly from GPIO, unless the module's current draw is verified GPIO-safe.

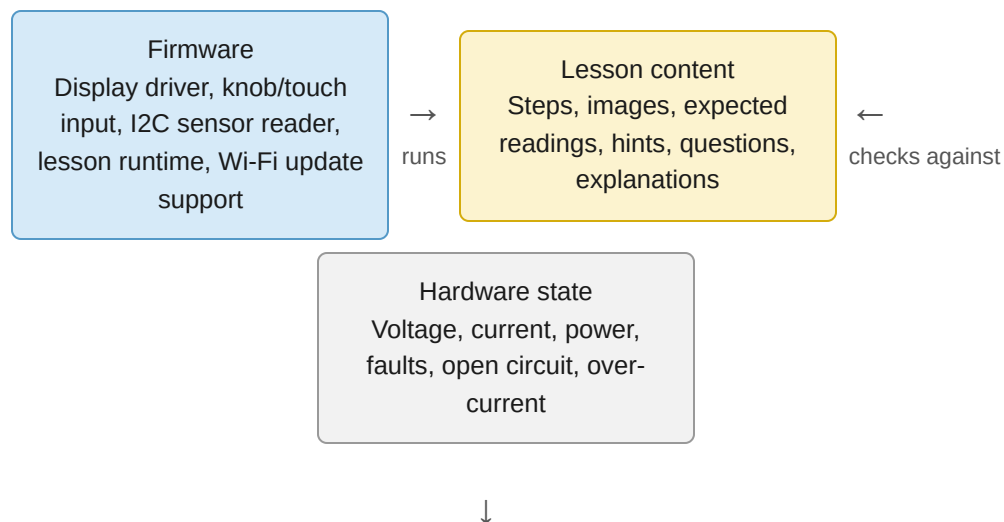
16	Storage	Umirokin clear organizer boxes	2 pack, adjustable dividers	1 / Buy now	Keeps parts from becoming a loose bag.
17	Work surface	Anti-static silicone electronics mat	~16" x 12"	1 / Optional	Contains the bench. Not required.

7. What to buy first

- BusBoard BB830
- Treedix USB-C breadboard power module
- Plugable USB-C power meter
- Adafruit INA219 current sensor
- Qwiic/STEMMA cable kit (plus an adapter cable if the Elecrow connector turns out not to be Qwiic-native — Section 4.4)
- Silicone jumper wires
- Resistor kit
- LED kit
- Button kit
- 10K potentiometers
- Organizer box

Defer: capacitor kits, transistor kit, and buzzer can be bought now for a complete Stage 1 box, but are not necessary for the first two or three lessons.

8. Software loading and lesson content REVISED



drives

On-screen tutor
"Place this resistor. Measure here. Press when
done. Expected: 3–5 mA. Measured: 3.9 mA.
Continue."

Figure 4. Software architecture for the tutor and lesson packs (structure unchanged; schema and example reading updated for the 3.3V default rail).

Lesson schema **EXTENDED**

Extended with safety and measurement-precision fields so assumptions are data, not prose, and with a structural power-off-before-rewiring behavior:

```
id: L2_led_with_resistor
title: "LED with resistor"
supply_rail: 3v3
steps:
  - prompt: "Place a 330 ohm resistor from row A10 to A15."
    wait_for: knob_press
    requires_power_off_before_step: true
  - prompt: "Place the LED long leg at A15, short leg to the ground rail."
    wait_for: knob_press
    requires_power_off_before_step: true
  - prompt: "Turn power on."
    measurement_source: ina219
    expected_units: mA
    expected_range: [3, 5]
    warning_threshold: 6
    hard_fault_threshold: 25
    max_safe_current_mA: 25
    gpio_safe: true
    requires_supervision: false
    on_pass: { advance: true, message: "Correct. Press to continue." }
    on_fail:
      open_circuit: "No current detected. Check the ground rail connection."
      over_range: "Current is higher than expected. Check resistor value and LED polarity"
  hints:
    - "LED current is set mostly by the resistor, not the LED."
    - "Red or yellow LEDs work best at 3.3V; blue/green/white need more headroom."
  next: L3_change_the_resistor
```

Any step with `requires_power_off_before_step: true` makes the engine render a standard interstitial — "Disconnect power before changing this wiring. Press when power is disconnected." — before the step prompt, instead of each lesson author writing that instruction differently.

measurement_source distinguishes a human-read value (user_meter, usb_meter) from a software-read one (ina219, gpio_adc), which matters because the USB meter and the INA219 are not interchangeable (Section 9). warning_threshold / hard_fault_threshold / max_safe_current_mA separate "this looks off" from "this is dangerous" from "this lesson's safe ceiling," and are what L10's controlled fault drill manipulates without ever exceeding the circuit's real safe operating current.

Developer mode

Connect the Elecrow to a Mac or PC with a USB-C data cable. Use PlatformIO first if possible, or Arduino IDE if following Elecrow examples. Upload firmware over USB-C. This is how the first prototypes should be built.

User mode

Later, the device should update over Wi-Fi. A user would choose a lesson pack from a menu and the device would download content. This is not Stage 1 work.

Storage plan MADE CONCRETE

Do not buy a microSD card yet. The Elecrow listing specifies 16M flash, UART, I2C, and FPC expansion, but does not advertise a microSD slot. The v1 plan:

- Firmware uploaded over USB-C during development.
- Lesson content (the schema above) compiled into firmware or stored in internal flash using LittleFS or an equivalent ESP32 filesystem.
- Wi-Fi content updates deferred until the lesson engine is stable.
- External SD only if hardware inspection confirms a usable slot, or if an SPI SD module is added later.

Avoid excessive flash writes NEW

Live measurements (voltage/current readings during an active lesson) stay in RAM. Only lesson completion, fault summaries, and calibration records get persisted to flash. High-frequency writes of live sensor data would shorten flash life for no benefit, since nothing downstream needs a continuous log during normal operation.

Lesson engine

The correct architecture is a small lesson engine, not custom firmware per lesson. The engine knows how to display a step, draw a wiring diagram, wait for a knob press, ask a question, read the current sensor, compare expected values against the schema above, and advance.

9. Smart Power Module path REVISED

V0: Visible power

Use the Pluggable USB-C meter. The learner reads voltage, current, and watts manually. Sufficient for Ohm's law lessons.

Measurement scope caveat: the Pluggable meter measures the USB-C power path, not necessarily the exact branch current of a breadboard circuit. If the meter sits upstream of the breadboard power module, it may read total input current to the module, not just the LED load — for very small LED circuits, that reading can be coarse or include module overhead. The meter is excellent for visible V/A/W intuition; the INA219 is the correct path for software-readable, circuit-specific measurement. Avoid presenting V0 meter readings as more precise than they are in early Ohm's law lessons.

V1: Software-observed power

Wire the INA219 in series with the breadboard load, connected to the Elecrow via I2C, following the topology in Figure 3 (Section 5). The lesson engine reads current and voltage directly.

V1 fault behavior: on over-current or out-of-range read, the lesson engine (1) shows a full-screen red fault banner, (2) displays the explicit instruction "Disconnect power now," and (3) logs the fault event (timestamp and reading, persisted — not the continuous live stream, see Section 8). There is no automatic cutoff at V1 — the learner is the disconnect mechanism. L10 (Section 12) gives the learner deliberate, supervised practice with this exact banner before they can encounter it by accident.

V2: Protected smart module

Design a custom board with USB-C input, current limiting, measurement, fault reporting, and a safe disconnect circuit (e.g., a load switch IC gated by the fault condition). This becomes the commercial product core, and is the first version where "warn" can become "warn and cut."

V3: Row-aware smart breadboard

Only after the power module works, explore sensing rows or continuity. This is much harder and should not be the first product.

10. Recommended desk layout for v1

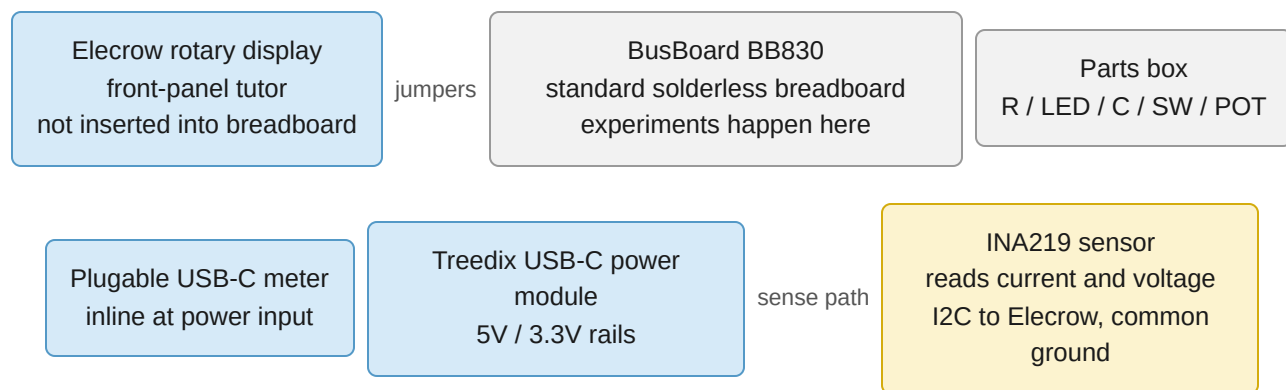


Figure 5. Recommended desk layout (unchanged from v1.0/v1.1).

11. Safety scope REVISED

- Stage 1 is low-voltage DC only: 3.3V (default) and 5V (isolated experiments only) breadboard circuits.
- No AC mains. No wall outlet experiments. No exposed lithium battery charging experiments.
- **The Elecrow / ESP32-S3 GPIO and I2C pins are 3.3V logic. Never connect 5V directly to a GPIO, I2C, UART, touch, display, or expansion pin unless the documentation explicitly states 5V tolerance.**
- Assume beginner mistakes. Use current limiting, start with higher resistor values, and disconnect power before rewiring — practiced directly in L1 and enforced structurally via `requires_power_off_before_step` (Section 8).
- The smart module should detect over-current and shut off or warn before damage occurs. At V1, the off-the-shelf module can only warn (Section 9); L10 gives the learner practiced, supervised exposure to that warning before it can happen unsupervised.

Current limits for early lessons NEW

Lesson	Target current	Note
L2: LED with resistor	< 5 mA at 3.3V	Red/yellow LED, 330Ω–1k resistor.
L3: Resistor comparison	< 10 mA	1k / 470Ω / 330Ω at 3.3V, worst case.
L4/L5: Series & parallel	< 20 mA total	Sum across all branches.
L8: GPIO-driven LED	well under GPIO abs. max	ESP32-S3 GPIO absolute max is well above this; size the resistor conservatively (target <10 mA) regardless.

L11: Buzzer/transistor	transistor- switched only	Do not drive a buzzer directly from a GPIO pin unless its current draw is verified GPIO-safe.
---------------------------	------------------------------	---

12. First guided lesson sequence RENUMBERED L0-L11

L0: First boot validation — before any breadboard wiring: load a vendor example over USB-C and confirm, in order: display output, capacitive touch response, knob rotation, knob press, and a successful USB-C firmware upload. Record the exact board definition, library versions, and example sketch name used — this is the known-good baseline for every lesson after it.

L1: Power-off habit — the screen asks the learner to disconnect power, then walks through placing/removing a single jumper, then reconnect. No measurement yet. Converts the Section 11 safety assumption into a practiced habit before it matters.

L2: LED with resistor: Build a simple LED circuit at 3.3V with a red or yellow LED. The screen teaches polarity, current limiting, and expected current (target <5 mA — see Section 11 table).

L3: Change the resistor: Swap 1k, 470Ω, and 330Ω resistors at 3.3V. The screen asks the learner to predict brightness and current before measuring.

L4: Series resistance: Place two resistors in series and measure lower current.

L5: Parallel branches: Build two LED branches and observe total current increase.

L6: Button input: Wire a pushbutton and pull-down resistor. The display changes when the button is pressed.

L7: Potentiometer: Use a 10K pot as a voltage divider. Turn the pot and watch analog voltage change.

L8: Software-controlled LED: Use the Elecrow to drive an LED through a resistor. Then map knob position to brightness.

L9: Smart measurement: Insert the INA219 into the load path (Figure 3). The display compares expected and measured current.

L10: Controlled fault-screen drill REVISED FROM "INDUCED FAULT" — uses the same safe circuit already on the board, well within the Section 11 current-limit table. The lesson temporarily lowers its own `warning_threshold/hard_fault_threshold` far below the circuit's actual safe current, so the existing, unmodified, safe current trips the red fault banner. No wiring change, no near-short, no real over-current is ever created. The learner practices reading the banner and the "disconnect power now" response without the circuit exceeding Section 11's limits.

L11: Transistor switch: Use a transistor to drive a buzzer or higher-current LED path from a GPIO pin (transistor-switched, not direct GPIO drive — see Section 11 table).

13. Example on-screen lesson flow UPDATED FOR 3.3V

1. Screen: Find a 330Ω resistor and a red or yellow LED. Press the knob when found.
2. Screen: Place one end at row A10 and the other at row A15.
3. Screen: Place LED long leg at A15 and short leg to the ground rail.
4. Screen: Disconnect power before connecting the 3.3V rail through the resistor path (structural interstitial, Section 8).
5. Screen: Turn power on. Expected current is about 3 to 5 mA at 3.3V with a red or yellow LED.
6. Screen, V0: Read the Plugable meter and enter or select the closest current (remember: this reads the USB-C input path, Section 9).
7. Screen, V1: Sensor reads 3.9 mA. Correct. Press to continue.
8. Screen: Replace the 330Ω resistor with 1k. Predict whether the LED gets brighter or dimmer.

14. Product concept

What is distinctive

Most electronics kits give instructions. This lab gives feedback. The device does not merely say what to build. It observes voltage and current and tells the learner what likely happened.

Why this can be a product

The first product does not require a proprietary breadboard. The first sellable unit could be a smart USB-C power and measurement module, plus an ESP32 display tutor, plus a standard BB830.

Product realism note **NEW** : the likely first sellable product is not a smart breadboard. It is a smart USB-C power and measurement module, paired with a standard BB830, paired with an ESP32 display tutor, with lessons as structured content packs (Section 8's schema). This keeps the product path practical and is the reason V3 row-aware breadboard sensing (Section 9) stays explicitly deferred rather than becoming the starting point.

Course packs

The same hardware could support course packs: Electronics Fundamentals, Internet Radio, Amateur Radio, SDR Basics, Sensors, Home Automation, and Robotics. With the lesson schema in Section 8, each course pack is a set of schema files plus assets, not a firmware fork.

Commercial risk

The difficult parts are not the first hardware. The hard work is curriculum design, safe fault handling, reliable connecting, and making lessons resilient when beginners miswire circuits. The lesson

schema and the V1 fault behavior (Sections 8–9) are concrete artifacts against that risk, not just statements of it.

15. Next build action after parts arrive REVISED

1. **Photograph and identify the Elecrow connector set** that arrives in the box. Identify the exact connector type and pinout. Confirm 3.3V, GND, SDA, and SCL pins before connecting the INA219. If no clean Qwiic/STEMMA-native connector is included, add the appropriate breakout/adaptor cable to the BOM before proceeding (Section 4.4).
2. **Run L0's first-boot validation** (Section 12): load a basic Elecrow example over USB-C, confirm display, touch, knob rotation, knob press, and USB-C upload. Record the board definition, library versions, and example sketch name.
3. Confirm whether the Elecrow exposes I2S pins for audio (Section 4.1), before Stage 2b BOM is finalized.
4. Assemble the BB830 rails with the Treedix power module. Verify 5V and 3.3V with the multimeter before connecting the Elecrow.
5. Build the first LED/resistor lesson (L2) manually using the USB-C meter for visible feedback, at 3.3V with a red or yellow LED.
6. Add the INA219 and make the Elecrow display current and voltage, following the topology in Figure 3. That is the first true Smart Lab milestone.
7. Draft the Stage 3 control-interface message schema (Section 4.2) as a one-page note, so Stage 2 firmware's event handling already emits in that shape.

16. Sourcing references

These are retailer listings, not the spec of record (see the Interface/Spec column in Section 6). Prices, stock, and listing details change; if a link goes stale, search by ASIN or product name.

- **Elecrow 2.1" rotary display** (ASIN B0G3TZGRC4): rotary knob, capacitive touch, ESP32-S3, Wi-Fi/BLE, UART, I2C, FPC expansion, 16M flash, 8M PSRAM, 5V/1A input. **Connector pinout and I2S availability not yet confirmed against the physical board — first build action (Section 15).**
- **BusBoard BB830** (ASIN B0040Z4QN8): 830 tie points, 4 power rails, 6.5 x 2.2 x 0.3 in, 36V/2A rating, 21–26 AWG insertion wire size.
- **Treedix USB-C breadboard power module** (ASIN B0CN6HYCJD): USB-C charging/power, 3.3V and 5V outputs.
- **Plugable USB-C power meter** (ASIN B0B5W5NKKN): voltage and amperage measurement, power monitoring, bidirectional connection, data passthrough.

- **Adafruit INA219** (ASIN B09CBSLXN7): high-side current measurement, 26V target voltage, $\pm 3.2\text{A}$ range, I2C address options.
- **Adafruit INA260** (official page): higher-capacity future option, integrated shunt, I2C, 3V/5V logic compatible, up to 36V/15A. Candidate for the V2 custom Smart Power Module.